

— COMP4052 • FINAL PROJECT

MimbleWimble Confidential Transactions

Hiding transaction amounts on a blockchain while still proving no value is created — explained from first principles, implemented on-chain in Solidity, and cross-validated against the Grin reference.

Ege Deniz • 220304118 • Introduction to Blockchain & DLT

 01 • THE PROBLEM

Public blockchains expose every amount.

On Bitcoin, the ledger is fully transparent: anyone can read who paid whom and exactly how much. For real payments — salaries, business flows — that is unacceptable. The challenge: hide the amounts, but keep the chain **publicly verifiable** so no one can cheat.

 02 • THE PRIVACY LANDSCAPE

Three ways to add privacy.

Monero

Ring signatures

Hides sender, receiver, and amount with ring signatures, stealth addresses, and RingCT. Larger transactions.

Zcash

zk-SNARKs

Hides everything with zero-knowledge proofs — powerful, but historically needed a trusted setup.

MimbleWimble

Commitments

Hides amounts with homomorphic Pedersen commitments. No addresses, no trusted setup, compact chain.

THE SOLUTION

 03 • MIMBLEWIMBLE

A blockchain where amounts
are **hidden**, yet every
transaction stays
mathematically verifiable.

Proposed anonymously in 2016 by "Tom Elvis Jedusor"; formalised by Andrew Poelstra. Live in production as Grin and Beam. The next slides build it from four ideas.

— 04 • IDEA 1 — HIDE THE AMOUNT

Pedersen commitment.

A commitment is a **locked box** for a number. The amount v is locked behind a random **blinding factor** r . **Hiding:** r is uniform, so the box reveals nothing about v . **Binding:** you cannot later open the box to a different amount, because no one knows how the two generators G and H relate.

$$C = v \cdot H + r \cdot G$$

— 05 • IDEA 2 — CHECK THE BALANCE

Commitments add up.

Pedersen commitments are **homomorphic** — adding two boxes gives the box of the sum. So the network can confirm **money in = money out + fee** by adding the commitments. If it balances, every amount term cancels and only a blinding term (the **excess**) remains. **No amount is ever revealed.**

$$\begin{aligned}\Sigma C_{in} - \Sigma C_{out} - \text{fee} \cdot H \\ = \text{excess} \cdot G\end{aligned}$$

— 06 • IDEA 3 — PROVE NO INFLATION

The kernel signature.

The leftover **excess**·**G** is published as a public key. The spender signs it with a **Schnorr signature**. A valid signature proves two things at once: the amounts **balanced**, and the spender **owns the inputs**. An unbalanced transaction would leave an amount term the spender cannot sign for.

verify: $s \cdot G == R + e \cdot P$

— 07 • IDEA 4 — BLOCK THE CHEAT

Range proofs.

The balance check accepts **negative amounts** — a spender could commit to -5 and +100, "balance" with +95, and mint 5 from nothing. A **range proof** proves each amount lies in $[0, 2^{64})$ without revealing it, closing the attack. This project uses the classical bit-commitment construction (pre-Bulletproofs).

 08 • WHY IT IS SECURE

Two generators, one hard problem.

- ◆ **secp256k1, the same curve as Bitcoin.** G is the standard generator; H is a second generator.
- ◆ **H is "nothing up my sleeve".** Derived from a public seed by try-and-increment, so nobody knows a scalar k with $H = k \cdot G$ — which is exactly what keeps commitments binding.
- ◆ **Everything rests on the discrete-log problem.** Given $Q = k \cdot G$, recovering k is infeasible. Hiding, binding, and signature unforgeability all follow.

— 09 • WHAT I BUILT

An on-chain verifier, a prover, and a reference.

Solidity

On-chain verifier

Verifies the kernel signature and commitment balance on Ethereum, with secp256k1 implemented from scratch.

IN SOLUTION

Python

The prover

Builds confidential transactions — commitments, range proofs, kernel signatures — and verifies them off-chain.

Grin

The reference

grinventions/mimblewimble-py. Identical inputs through it and this project must agree.

 10 • THE PROPOSED SOLUTION

Verifying privacy on Ethereum.

- ◆ `verifyKernelSignature` — confirms the spender knows the kernel excess (ownership).
- ◆ `verifyBalance` — confirms $\Sigma \text{ inputs} - \Sigma \text{ outputs} - \text{fee} \cdot H$ equals the kernel excess (no inflation).
- ◆ `secp256k1` from scratch — point add, double, scalar-mul, modular inverse, since the EVM has no curve precompile.
- ◆ Range proofs stay off-chain — on-chain they would cost tens of millions of gas; verified in the Python prover instead.

— 11 • INSIDE THE VERIFIER

The no-inflation check, in code.

Sum the input points, subtract each output point, subtract the fee term, and compare the result to the published kernel excess. If the amounts did not balance, an amount term would remain and the equality would fail.

```
uint256 x = inX[0]; uint256 y = inY[0];
for (i=1; i<inX.length; i++)          // add inputs
    (x,y) = ecAdd(x,y, inX[i],inY[i]);
for (i=0; i<outX.length; i++)          // subtract outputs
    (x,y) = ecAdd(x,y, outX[i],(PP-outY[i])%PP);
if (fee != 0) {                          // subtract fee·H
    (fx,fy) = ecMul(HX,HY,(NN-fee)%NN);
    (x,y)   = ecAdd(x,y, fx,fy);
}
return x==kernelExcessX && y==kernelExcessY;
```

— 12 • THE LIVE DEMO

One command runs everything.

```
$ bash scripts/demo.sh
```

```
1/4 Part A → ALL SCENARIOS AGREE
```

```
2/4 Verify → [OK] [OK] [OK]
```

```
3/4 Tamper → REJECTED
```

```
4/4 On-chain → 10 passing
```

- ◆ **Part A** — reference vs mine on 5 inputs; verdicts must match.
- ◆ **Verify** — a real confidential transaction passes all checks.
- ◆ **Tamper** — flip one byte → it is rejected.
- ◆ **On-chain** — the Solidity verifier on Ethereum.

 13 • VERIFICATION

Cross-validated against Grin, and fully tested.

```
# reference vs this implementation, the instructor's inputs
$ python scripts/compare_with_reference.py --scenarios inputs.json
Result: ALL SCENARIOS AGREE

$ pytest tests/      → 21 passed
$ npx hardhat test   → 10 passing
```

 14 • SCOPE & FUTURE WORK

Known limitations.

- ◆ **Cut-through & kernel aggregation** — not implemented; this is the CT primitive layer.
- ◆ **Bulletproofs** — would replace the classical range proof and shrink it roughly ten-fold.
- ◆ **Gas optimization** — affine coordinates kept for clarity; Jacobian + wNAF would cut on-chain cost.
- ◆ **No P2P layer** — the focus is the cryptographic core, not networking.

 15 • REFERENCES

Foundations.

Jedusor (2016) – the original MimbleWimble text.

Poelstra (2016) – the canonical MimbleWimble write-up.

Maxwell – Confidential Transactions.

Grin documentation – docs.grin.mw

grinventions/mimblewimble-py – reference implementation.

MimbleWimble Confidential Transactions • Ege Deniz • COMP4052